GPU-BASED SATELLITE ORBIT FEASIBILITY SIMULATOR USING CUDA

Synopsis

PCAP MINOR PROJECT

Submitted By

Divyanshu Gupta, Reg No: 230962274
Kumar Satyam, Reg No: 230962226
Dhruv Saraogi, Reg No: 230962250

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY,
MANIPAL ACADEMY OF HIGHER EDUCATION

***Under the guidance of*: Dr G Arul Elango**

Date of submission: **21st March 2026**

I. Introduction

With the rapid growth of space exploration and satellite-based technologies such as communication, navigation, and Earth observation, understanding the **stability and feasibility of satellite orbits** has become a critical aspect of aerospace engineering. The motion of satellites is governed by fundamental laws of physics, primarily Newton's law of gravitation and laws of motion, and is highly sensitive to initial conditions such as velocity, altitude, and position.

Small variations in these parameters can significantly affect whether a satellite maintains a stable orbit, escapes Earth's gravitational field, or re-enters the atmosphere. Therefore, analyzing multiple possible trajectories is essential for mission planning and optimization.

Traditional simulation approaches rely on CPU-based computations, which become increasingly inefficient when evaluating a large number of satellite trajectories due to their high computational complexity and sequential execution nature. As the number of simulated satellites increases, the time required for computation grows rapidly, making large-scale analysis impractical.

To address this limitation, this project proposes a **GPU-accelerated simulation framework using CUDA**, which leverages the massive parallel processing capabilities of modern GPUs. By assigning each satellite simulation to an individual thread, the system can simulate 5000–6000 satellites simultaneously, significantly reducing computation time.

This approach enables faster, scalable, and efficient analysis of orbital behavior, making it highly suitable for applications in satellite mission design, aerospace simulations, and real-time feasibility analysis.

II. Literature Review

The study of satellite motion and orbital dynamics is deeply rooted in classical mechanics and aerospace engineering principles. Foundational works such as those by Goldstein [6], Curtis [5], and Anderson [7] describe the mathematical modeling of motion under gravitational forces and provide the theoretical basis for orbital simulations. Numerical integration techniques such as Euler and Runge-Kutta methods are widely used to approximate solutions to the differential equations governing satellite motion.

Traditional implementations of orbital simulations are typically executed on CPUs, which limits their scalability when handling large datasets or performing repeated simulations with varying initial conditions. As the number of simulated trajectories increases, these approaches become computationally expensive and inefficient.

Recent advancements in parallel computing have enabled the use of GPUs for accelerating such simulations. CUDA, developed by NVIDIA, provides a powerful framework for implementing parallel algorithms on GPUs [1], [13]. Several studies and textbooks, including works by Sanders and Kandrot [2], Wilt [3], and Kirk and Hwu [4], demonstrate how GPU-based programming can significantly improve performance for computationally intensive tasks.

In the domain of physics simulations, GPU acceleration has been successfully applied to problems such as N-body simulations and particle interactions. Systems like GENGA [8] and Swarm-NG [9] utilize CUDA to perform large-scale gravitational computations efficiently, achieving substantial speedups compared to CPU-based implementations. Similar acceleration techniques have also been applied in other simulation domains, including Monte Carlo simulations [11] and fluid dynamics [12].

More recent research has explored GPU-based trajectory propagation and orbital simulations, highlighting the potential for high-performance analysis of satellite motion [10]. These works demonstrate the effectiveness of GPU computing in handling large-scale simulations with improved accuracy and efficiency.

However, despite these advancements, there is limited work focused on integrating large-scale parallel satellite simulations with automated classification of orbital feasibility (i.e., stable orbit, escape, or crash). This gap highlights the need for a system that not only performs high-speed simulations but also provides meaningful classification and analysis, which this project aims to address.

III. Problem Statement

Determining the feasibility of satellite orbits is a computationally intensive problem that involves evaluating the motion of satellites under gravitational forces over time. For a given set of initial conditions, a satellite may achieve a stable orbit, escape Earth's gravitational influence, or fall back to Earth.

When this evaluation needs to be performed across thousands of satellites with varying initial velocities and altitudes, the computational demand increases significantly. Traditional CPU-based approaches are inefficient for such large-scale simulations due to their limited parallel processing capabilities.

Moreover, existing systems often focus only on trajectory simulation without providing an efficient mechanism for classification of orbital outcomes, which is essential for decision-making in real-world applications.

Therefore, there is a need to develop a system that can efficiently simulate multiple satellite trajectories, utilize parallel computing for faster execution, and automatically classify the outcomes of each simulation.

IV. Objectives

The key objectives of this project are:

- To simulate satellite motion under Earth's gravitational influence using numerical methods.
  - Model gravitational forces based on Newton's law of gravitation.
  - Implement time-stepped numerical integration (e.g., Euler method).
- To analyze the impact of varying initial conditions on orbital behavior.
  - Study the effect of different initial velocities on trajectory stability.

  - Evaluate the role of altitude and initial position in determining orbit feasibility.
- To classify satellite trajectories into different outcome categories.
  - Identify stable orbits where satellites remain bounded around Earth.
  - Detect escape trajectories where satellites exceed escape velocity.
  - Identify crash scenarios where satellites re-enter Earth's atmosphere.
- To design and implement a CUDA-based parallel simulation system.
  - Map each satellite simulation to an individual GPU thread.
  - Optimize memory usage and parallel execution for large-scale simulations.
- To evaluate system performance and scalability.
  - Compare execution time between CPU and GPU implementations.
  - Analyze performance improvements with increasing number of satellites (5000–6000).
- To visualize simulation results for better interpretation.
  - Plot satellite trajectories in 2D/3D space.
  - Generate stability maps based on velocity and altitude variations.
- To extend the simulation for more realistic scenarios (future scope).
  - Incorporate perturbations such as atmospheric drag and external forces.
  - Explore multi-body interactions (e.g., Earth-Moon system).

REFERENCES

[1] NVIDIA Corporation, "CUDA C Programming Guide," 2023. [Online]. Available: https://docs.nvidia.com/cuda/

[2] J. Sanders and E. Kandrot, *CUDA by Example: An Introduction to General-Purpose GPU Programming*, Addison-Wesley, 2010.

[3] N. Wilt, *The CUDA Handbook: A Comprehensive Guide to GPU Programming*, Addison-Wesley, 2013.

[4] D. B. Kirk and W. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, Morgan Kaufmann, 2016.

[5] H. D. Curtis, *Orbital Mechanics for Engineering Students*, Butterworth-Heinemann, 2019.

[6] H. Goldstein, *Classical Mechanics*, Pearson Education, 2002.

[7] J. D. Anderson, *Introduction to Flight*, McGraw-Hill Education, 2015.

[8] S. L. Grimm and J. G. Stadel, "The GENGA Code: Gravitational Encounters in N-body Simulations with GPU Acceleration," *Astronomy & Computing*, 2014.

[9] S. Dindar et al., "Swarm-NG: A CUDA Library for Parallel N-body Integrations," *arXiv preprint arXiv:1208.1157*, 2012.

[10] A. Masat et al., "GPU-based Augmented Trajectory Propagation for Orbital Simulations," 2023.

[11] J. Spiechowicz et al., "GPU Accelerated Monte Carlo Simulations with CUDA," *arXiv*, 2014.

[12] M. Januszewski and M. Kostur, "Sailfish: Multi-GPU Fluid Simulation Using CUDA," *arXiv*, 2013.

[13] NVIDIA, "CUDA Programming Model and GPU Architecture Overview," 2023.